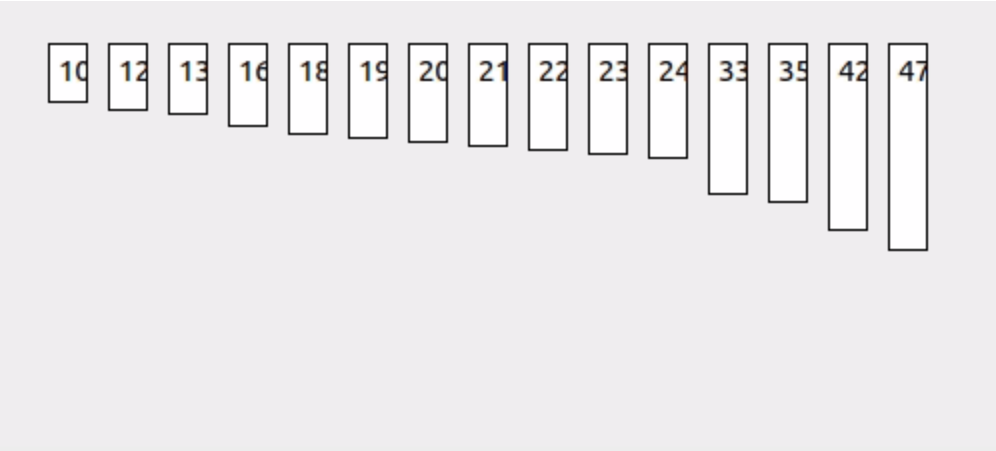
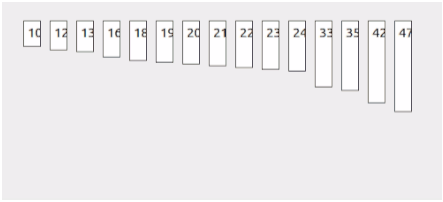
Interpolation search

en.wikipedia.org/wiki/Interpolation_search



Interpolation search is an algorithm for searching for a key in an array that has been ordered by numerical values assigned to the keys (*key values*). It was first described by W. W. Peterson in 1957.[1] Interpolation search resembles the method by which people search a telephone directory for a name (the key value by which the book's entries are ordered): in each step the algorithm calculates where in the remaining search space the sought item might be, based on the key values at the bounds of the search space and the value of the sought key, usually via a linear interpolation. The key value actually found at this estimated position is then compared to the key value being sought. If it is not equal, then depending on the comparison, the remaining search space is reduced to the part before or after the estimated position. This method will only work if calculations on the size of differences between key values are sensible.



Visualization of the interpolation search algorithm in which 24 is the target value.

Class	Search algorithm
Data structure	Array
Worst-case performance	*O*(*n*)

Best-case performance	*O*(1)
Average performance	*O*(log(log(*n*)))[2]
Worst-case space complexity	*O*(1)
Optimal	Yes

Interpolation search

By comparison, binary search always chooses the middle of the remaining search space, discarding one half or the other, depending on the comparison between the key found at the estimated position and the key sought — it does not require numerical values for the keys, just a total order on them. The remaining search space is reduced to the part before or after the estimated position. The linear search uses equality only as it compares elements one-by-one from the start, ignoring any sorting.

On average the interpolation search makes about log(log(*n*)) comparisons (if the elements are uniformly distributed), where *n* is the number of elements to be searched. In the worst case (for instance where the numerical values of the keys increase exponentially) it can make up to O(*n*) comparisons.

In interpolation-sequential search, interpolation is used to find an item near the one being searched for, then linear search is used to find the exact item.

Performance

Using big-O notation, the performance of the interpolation algorithm on a data set of size *n* is *O*(*n*); however under the assumption of a uniform distribution of the data on the linear scale used for interpolation, the performance can be shown to be *O*(log log *n*).[3][4][5] However, Dynamic Interpolation Search is possible in *o*(log log *n*) time using a novel data structure.[6]

Practical performance of interpolation search depends on whether the reduced number of probes is outweighed by the more complicated calculations needed for each probe. It can be useful for locating a record in a large sorted file on disk, where each probe involves a disk seek and is much slower than the interpolation arithmetic.

Index structures like B-trees also reduce the number of disk accesses, and are more often used to index on-disk data in part because they can index many types of data and can be updated online. Still, interpolation search may be useful when one is forced to search certain sorted but unindexed on-disk datasets.

Adaptation to different datasets

When sort keys for a dataset are uniformly distributed numbers, linear interpolation is straightforward to implement and will find an index very near the sought value.

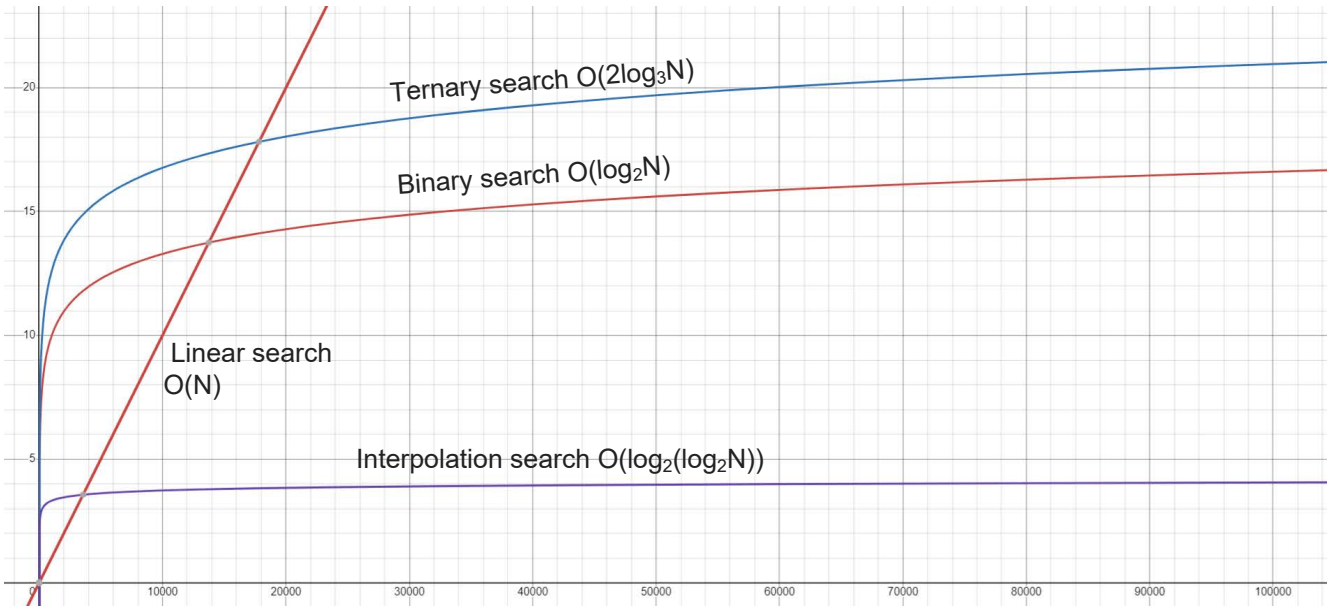
On the other hand, for a phone book sorted by name, the straightforward approach to interpolation search does not apply. The same high-level principles can still apply, though: one can estimate a name's position in the phone book using the relative frequencies of letters in names and use that as a probe location.

Some interpolation search implementations may not work as expected when a run of equal key values exists. The simplest implementation of interpolation search won't necessarily select the first (or last) element of such a run.

Book-based searching

The conversion of names in a telephone book to some sort of number clearly will not provide numbers having a uniform distribution (except via immense effort such as sorting the names and calling them name #1, name #2, etc.) and further, it is well known that some names are much more common than others (Smith, Jones,) Similarly with dictionaries, where there are many more words starting with some letters than others. Some publishers go to the effort of preparing marginal annotations or even cutting into the side of the pages to show markers for each letter so that at a glance a segmented interpolation can be performed.

Big O for various searches:



Note: linear search is displayed as (1/1000)x just to distinguish it from the y-axis!